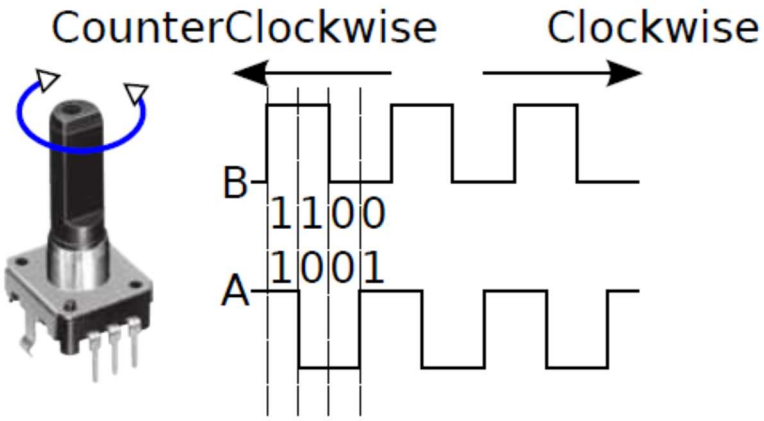
Lab11 - Encoder

1. Cos'è un encoder incrementale

Un **encoder incrementale** dà un **numero fisso di impulsi per giro (PPR, pulses per revolution)**. Non fornisce una posizione assoluta, ma solo gli spostamenti. Per ottenere la posizione **bisogna accumulare gli impulsi in un contatore, e all'avvio bisogna portare il sistema in una posizione di "home" per azzerare il contatore**. Alcuni encoder hanno anche un segnale "marker" (index), un impulso per giro, che qui non ti serve.
Con un solo canale hai gli impulsi ma non il verso. Per il verso servono due canali, A e B, sfasati di 90° (quadratura). Quale dei due è in anticipo dipende dal verso di rotazione. Se A sale mentre B è basso vai in un verso, se A sale mentre B è alto vai nell'altro. **Il circuito dentro il timer fa proprio questo confronto, in hardware, a ogni fronte.**



2. Le tre modalità del timer (x1, x2, x4)

Il timer ha una modalità encoder dedicata: il contatore CNT sale o scende da solo in base ai segnali su CH1 e CH2, senza CPU né interrupt. Le modalità sono tre:

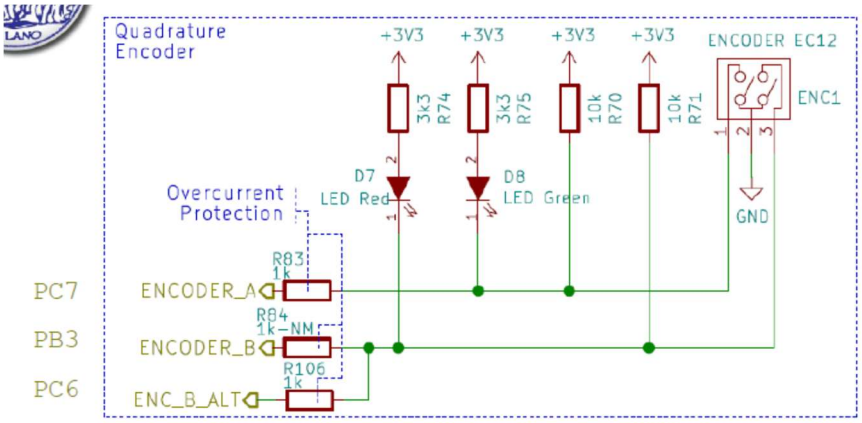
Mode	Fronti contati	Conteggi per passo meccanico	Conteggi per giro (encoder della scheda)
TI1	solo fronti di A (CH1), verso deciso dal livello di B	±2	±24
TI2	solo fronti di B (CH2), verso deciso dal livello di A	±2	±24
TI1 e TI2	tutti i fronti di A e B	±4	±48

Il "±" indica il verso: il contatore sale in un verso e scende nell'altro. Più fronti conti, più risoluzione ottieni.
La risposta alla domanda delle slide ("How many counts does a full rotation provide?") dipende dalla modalità scelta: **24** in TI1 o TI2, **48** in TI1 e TI2. Questo numero è la costante COUNTS_PER_REV che ti serve per convertire in rpm. Il valore effettivo dipende dall'encoder montato, quindi verificalo girando la manopola di un giro completo e leggendo il contatore.

3. Hardware della scheda (PMDB16)

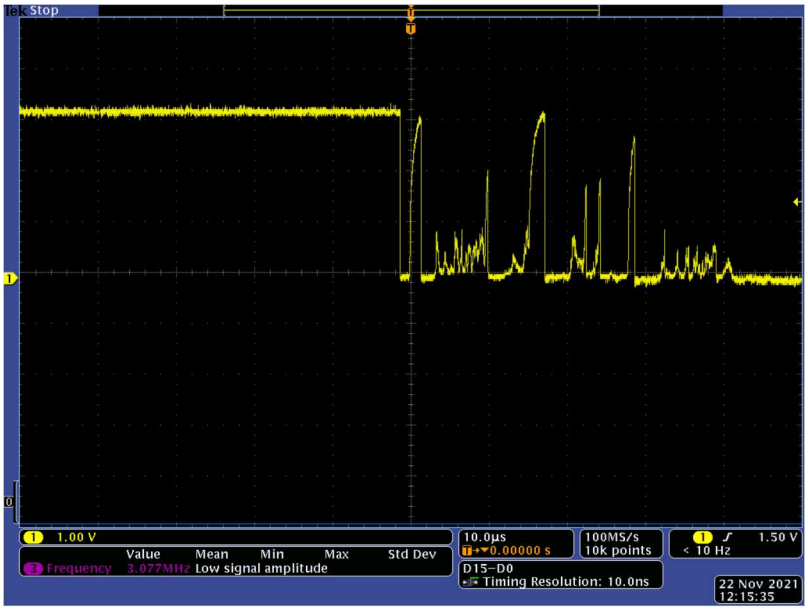
L'encoder è un EC12 con resistenze di pull-up esterne da 10 kΩ. I segnali passano da resistenze serie da 1 kΩ, che servono come protezione da sovracorrente. L'encoder è collegato ai pin **PC6 e PC7**, che sono TIM3_CH1

e TIM3_CH2. Questi pin possono essere pilotati solo da **TIM3**, quindi devi usare TIM3 per l'encoder. Su CubeMX i due pin vanno impostati come funzione alternata del timer, e in genere ci pensa da solo la scelta "Encoder Mode" del timer.
TIM3 è un timer a 16 bit, quindi il contatore va da 0 a 65535 (Counter Period = 65535).



4. Rimbalzi e filtro digitale

Sull'encoder non c'è debouncing hardware. I contatti meccanici rimbalzano per alcuni microsecondi (lo mostra l'oscilloscopio nelle slide). Senza filtro il timer conterebbe fronti spurii e la posizione andrebbe alla deriva.



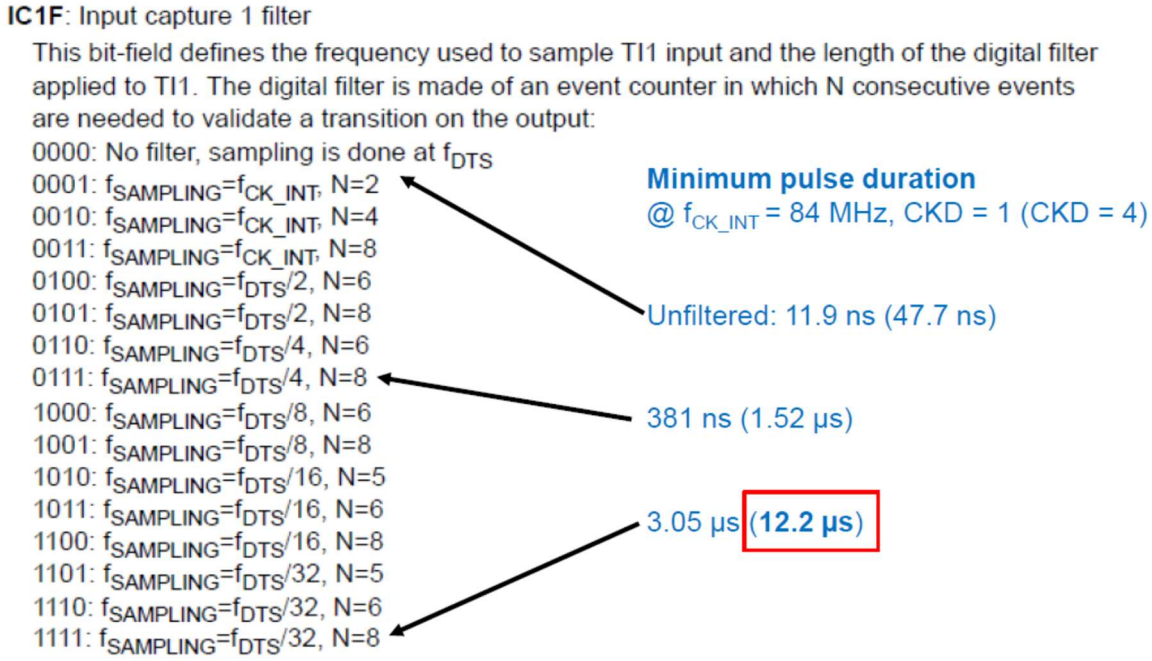
La soluzione è il **filtro digitale di ingresso** del timer, i campi **IC1F** e IC2F**. Il timer campiona l'ingresso a una frequenza f_SAMPLING e convalida una transizione solo se vede N campioni consecutivi uguali. Un impulso più breve di N periodi di campionamento viene scartato.** La frequenza di riferimento è:

$$f_{DTS} = \frac{f_{CK_INT}}{CKD}$$

dove CKD è il divisore dell'Internal Clock Division. Con il valore di filtro massimo (15, cioè f_DTS/32 con N=8) l'impulso minimo accettato è 8 × 32 / f_DTS:

- con CKD = 1 (f_DTS = 84 MHz) vale 3,05 µs
- con CKD = 4 (f_DTS = 21 MHz) vale **12,2 µs**

Le slide usano **CKD = 4** e **filtro = 15 (1111)**, cioè 12,2 µs, che copre tranquillamente i rimbalzi. Il compromesso è che un filtro troppo aggressivo taglia anche gli impulsi veri quando l'encoder gira molto veloce. Per una manopola girata a mano 12 µs sono un valore ampiamente sicuro.



5. Configurazione in CubeIDE (TIM3)

1. Timers → TIM3 → **Combined Channels = Encoder Mode**.
2. Counter Period = 65535, Internal Clock Division = **Division by 4**, auto-reload preload = Disable.
3. Nella sezione Encoder: Encoder Mode = TI1 (oppure TI1 and TI2 per la risoluzione x4). Per entrambi i canali IC Selection = Direct, Prescaler = No division, **Input Filter = 15**.
4. Polarity: nelle slide il canale 1 è Falling Edge e il canale 2 Rising Edge. Invertire la polarità di **un solo** canale inverte il verso di conteggio. È il modo più pulito per far corrispondere "orario" a "+" se all'inizio il segno risulta invertito.

6. Funzioni

1. HAL_TIM_Encoder_Start()

HAL_StatusTypeDef HAL_TIM_Encoder_Start(TIM_HandleTypeDef *htim,uint32_t Channel);

Avvia il **timer configurato in modalità Encoder**. In questa modalità il timer utilizza i segnali dell'encoder, normalmente collegati ai **Channel 1 e Channel 2** del timer, per incrementare o decrementare automaticamente il **Counter**.
Quando l'encoder viene ruotato, il timer aggiorna automaticamente il proprio contatore.
A seconda della direzione di rotazione il counter aumenta o diminuisce.
Quindi non devi necessariamente leggere manualmente i due segnali dell'encoder e determinare via software la direzione: **il timer hardware si occupa di questa operazione**.

```
HAL_TIM_Encoder_Start(&htim2, TIM_CHANNEL_ALL);
```

2. __HAL_TIM_GET_COUNTER()

__HAL_TIM_GET_COUNTER(TIM_HandleTypeDef *htim)

Legge il **valore attuale del Counter Register (CNT)** del timer. Nel caso di un encoder, questo valore rappresenta il conteggio degli impulsi/eventi generati dalla rotazione dell'encoder.
La usi quando vuoi **sapere la posizione attuale dell'encoder**, o più precisamente il conteggio accumulato dal timer.

Esempio completo

Un esempio molto semplice potrebbe essere:

```
/* Avvio del timer in modalità Encoder */
HAL_TIM_Encoder_Start(&htim2, TIM_CHANNEL_ALL);

while (1)
{
    int16_t position;

    /* Leggo il valore del contatore */
    position = __HAL_TIM_GET_COUNTER(&htim2);

    HAL_Delay(100);
}
```

La cosa importante è che **non devi incrementare manualmente position quando l'encoder ruota**.
È il timer che fa il lavoro.

Lab11_1

Lab11_1: leggi la posizione in rpm (rotations per minute) dell'encoder rotazionale e mandala ogni secondo ad un remote terminal tramite UART in DMA. Usa "+" per verso orario e "-" antiorario.

File .ioc

Pin

- PC6 = TIM3_CH1
- PC7 = TIM3_CH2
- PA2 = USART_TX
- PA3 = USART_RX

Timers

- Timer 2 (1Hz):
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=839999

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)(99+1)} = 1\ Hz$$

- Timer 3 (Encoder):
  - Combined Channels: Encoder Mode
  - CKD: 4
  - Encoder:
    - Polarity: Falling Edge
    - Input Filter: 15

Connectivity

- USART2:
  - Mode: Asynchronous
  - Baud Rate: 115200
  - Word Length: 8
  - Parity: None
  - Stop Bits: 1
  - DMA:
    - Request: TX (Memory to Peripheral)
    - Priority: High
    - Mode: Normal
  - NVIC:
    - DMA stream6 global interrupt = Enabled
    - USART2 global interrupt = Enabled

Codice

Private Define

```
#define STEPS_PER_REVOLUTION 24
```

Private Variables

```
char tx_buffer[50];
```

```c
float rpm;
static uint16_t value;
static uint16_t past_value=0;
```

Timer Callback (1Hz)

```c
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
      // Per osservare quanti "step" corrispondono ad una rivoluzione completa
      // Così abbiamo letto CNT = 24
      /*uint16_t curr = __HAL_TIM_GET_COUNTER(&htim3);
      int len = sprintf(tx_buffer, "CNT = %u\r\n", curr);
      HAL_UART_Transmit_DMA(&huart2, (uint8_t*)tx_buffer, len);*/

      value = __HAL_TIM_GET_COUNTER(&htim3);
      int16_t delta = (int16_t) (value-past_value);
      rpm = ((float)delta/STEPS_PER_REVOLUTION)*60.0f;
      int len = sprintf(tx_buffer, "rpm = %+.1f \r\n", rpm);
      HAL_UART_Transmit_DMA(&huart2, (uint8_t*)tx_buffer, len);
      past_value=value;
    }
}
```

Inizializzazioni

```c
HAL_TIM_Encoder_Start(&htim3, TIM_CHANNEL_ALL);
HAL_TIM_Base_Start_IT(&htim2);
```

Overflow e underflow del contatore. Il contatore è a 16 bit e va a capo: da 65535 passa a 0 girando in un verso, e da 0 passa a 65535 girando nell'altro. Una differenza normale tra due letture darebbe un valore enorme e sbagliato. Il trucco è fare la sottrazione in modulo 2^{16} e interpretarla con segno, cioè (int16_t)(curr - prev). Esempio con overflow: prev = 65530 e curr = 10, quindi il contatore si è spostato di +16, e il cast dà proprio +16. Con underflow: prev = 5 e curr = 65531 danno −10, come atteso. Il metodo funziona finché tra due letture ti sposti di meno di 32768 conteggi, cosa sempre vera con una manopola e finestra di 1 s. Ricorda l'ordine "adesso − precedente", mai il contrario.

Risoluzione della misura. Con una finestra di 1 s e 48 conteggi per giro, un singolo conteggio vale 60/48 = 1,25 rpm. La velocità che vedi è quindi quantizzata a multipli di 1,25 rpm. Con 24 conteggi per giro il passo diventa 2,5 rpm. Una finestra più lunga o una modalità x4 migliorano la risoluzione.